

Monitoramento e otimização de recursos em aplicativos Flutter

Beatriz Vocurca Frade

Universidade Federal de Minas Gerais
Engenharia de Sistemas
Orientador: Prof. Ramon Lacerda Marques
2026



Problema e motivação

- Flutter acelera o desenvolvimento multiplataforma, mas não elimina gargalos de desempenho.
- Apps modernos combinam listas dinâmicas, animações, rede, estado reativo e dispositivos muito diferentes.
- Problemas aparecem como *jank*, queda de FPS, alto uso de memória, chamadas de rede excessivas e menor duração de bateria.

Pergunta de pesquisa

Como tornar o diagnóstico de desempenho mais acessível, contínuo e acionável em Flutter?

Motivação prática

A primeira leitura do problema deve acontecer enquanto o aplicativo ainda está sendo construído.

Dimensão humana

Ideia central

Desempenho não é só número: ele afeta a experiência, o dispositivo e a confiança de quem usa o aplicativo.

Sintomas técnicos

jank, espera, rede
memória e bateria

Evidência compreensível

métricas e alertas
recomendações no app

Decisão responsável

priorizar correções
sustentar escolhas

Pessoa usuária

Fluidez, espera, bateria,
aquecimento e confiança.

Desenvolvimento

Ler sinais no app e decidir
o foco.

Recursos

Uso responsável de CPU,
memória, rede e bateria.

Aproximar evidência técnica de uma decisão compreensível e centrada na pessoa.

Objetivos

Objetivo geral

Desenvolver e avaliar funcionalmente o pacote `collector_flutter`, uma biblioteca Dart/Flutter embarcada para coletar métricas, analisar sintomas e apresentar recomendações em tempo real.

- Coletar FPS, *jank*, memória, rede, CPU, bateria, eventos e *rebuids*.
- Organizar a solução em arquitetura modular.
- Exibir métricas e recomendações no próprio app.
- Gerar recomendações heurísticas por severidade.
- Exportar sessões em JSON e *frame timings* em CSV.
- Validar o comportamento em execuções funcionais.

Base científica e lacuna

Literatura sobre desempenho móvel

- Pathak, Hoque e Li: energia depende de CPU, rede, APIs, execução e memória.
- Rua e Saraiva; Nawrocki: desempenho móvel é multidimensional.
- AppInsight: monitorar em execução ajuda a localizar gargalos reais.

O que a base sustenta

Observar FPS, *jank*, memória, rede, CPU, bateria e *rebuid*s em conjunto, não como métricas isoladas.

Ferramentas existentes

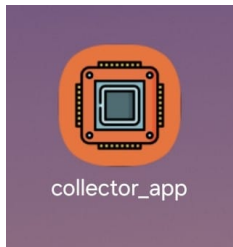
- *Flutter DevTools*, Android Profiler, Perfetto e Instruments coletam dados profundos.
- Em geral exigem ferramenta externa, configuração e interpretação manual.
- Firebase atua mais em produção, com menor detalhe de renderização e CPU.

Lacuna identificada

Coleta, interpretação e recomendação ainda aparecem separadas no fluxo diário de desenvolvimento Flutter.

Proposta: biblioteca desenvolvida

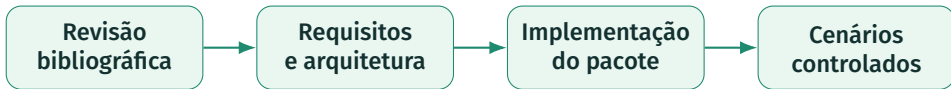
- O pacote `collector_flutter` é integrado à aplicação monitorada.
- A ferramenta proposta coleta métricas durante a execução do app.
- A análise heurística classifica sintomas e gera recomendações por severidade.
- O dashboard mostra sinais de desempenho sem exigir troca imediata para uma ferramenta externa.



Ideia central

Aproximar o primeiro diagnóstico do fluxo real de desenvolvimento Flutter.

Metodologia



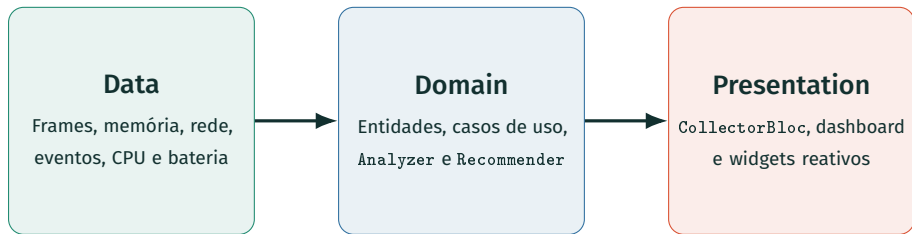
Tipo de avaliação

Validação funcional em app de exemplo, com observação de sinais técnicos induzidos.

Critério de leitura

Painel, logs, capturas, exportação e ADB foram usados como evidências qualitativas de rastreabilidade.

Arquitetura em camadas



Por que isso importa

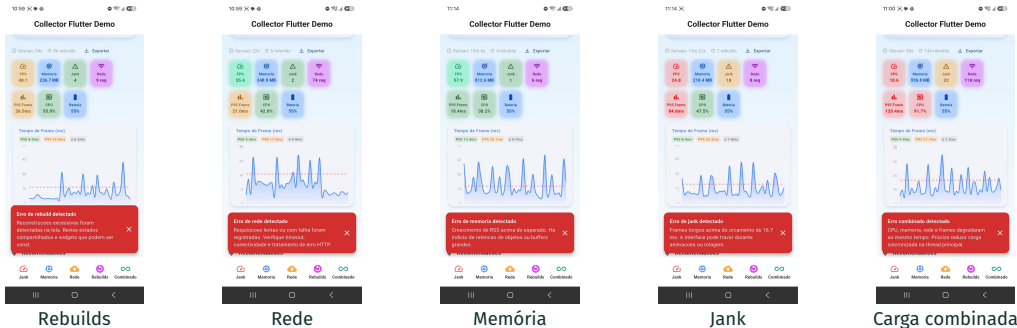
Separar as camadas facilitou evoluir CPU, bateria, exportação e testes sem misturar responsabilidades.

Resultados observados

Cenário	Sinais registrados	Leitura
Rebuilds	30 <i>rebuilds</i> , FPS 55,4, P95 20,4 ms e 2 frames longos.	A carga de interface foi registrada pelo painel.
Rede	9 requisições, FPS 42,9, P95 139,5 ms e 4 frames longos.	O cliente instrumentado registrou eventos e latências.
Memória	ADB: RSS 236.480 KB e PSS 222.232 KB.	Houve aumento real de memória; o snapshot do painel mostrou limite de leitura interna.
Combinado	17 requisições, 54 <i>rebuilds</i> , FPS 44,6, P95 37,7 ms e CPU 75,3%.	Múltiplos sinais apareceram juntos e geraram recomendações compatíveis.

Os valores vêm dos registros documentados na monografia: painel, logs, capturas e leituras ADB.

Evidências da validação funcional



Painel e capturas

Comportamento visual preservado no apêndice.

Logs e exportação

Valores e sessões rastreáveis fora do app.

ADB e testes

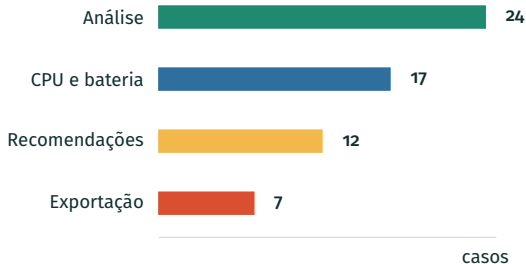
Apoio qualitativo e contratos internos verificados.

Testes automatizados

60 testes

Papel dos testes

Eles verificam cálculos, serialização, recomendações, exportação e *fallbacks* dos canais nativos.



Os testes complementam execuções manuais e fortalecem contratos internos da implementação.

Publicação no pub.dev

Disponível para a comunidade

O pacote foi publicado no repositório oficial do ecossistema Dart/Flutter, permitindo instalação direta por outros desenvolvedores e pesquisadores.

Endereço

pub.dev/packages/
collector_flutter

versão 0.1.3
32 pessoas usando

We analyzed this package 5 hours ago, and awarded it 150 pub points (of a possible 160):

✓ Follow Dart file conventions	30/30	⌵
✓ Provide documentation	20/20	⌵
✗ Platform support	10/20	⌵
✓ Pass static analysis	50/50	⌵
✓ Support up-to-date dependencies	40/40	⌵

Impacto prático

A publicação reforça reprodutibilidade, acesso aberto e possibilidade de evolução pela comunidade.

Próximos trabalhos

Desempenho

Diagnóstico acessível, contínuo e acionável.

Ferramenta integrada

Sintomas técnicos no app.

Ferramenta externa

Averiguar achados com *Flutter DevTools*, *Perfetto* e *Android Profiler*.

Validação

Ampliar avaliação funcional e qualitativa em perfis reais.

Testes

Expandir a suíte automatizada e explicitar o limite metodológico.

Conclusão

- A biblioteca desenvolvida aproxima o primeiro diagnóstico de desempenho do fluxo real de desenvolvimento Flutter.
- A arquitetura em camadas sustentou coleta, análise, dashboard, exportação, CPU, bateria e testes.
- A avaliação foi funcional e qualitativa: painel, logs, capturas, ADB e 60 testes.
- A publicação no pub.dev amplia o acesso ao pacote e permite que a comunidade teste, avalie e evolua a ferramenta.

Mensagem final

Performance não é apenas fazer o app rodar mais rápido; é fazer o app respeitar melhor o dispositivo, a bateria e a pessoa que o usa.

Perguntas?

O pacote `collector_flutter`

UF *m* G



Beatriz Vocurca Frade | Engenharia de Sistemas – UFMG